

Project 3 Report

Matrix Multiplication in C and Performance Optimization

1 Project Topic

This project implements dense matrix multiplication in C and compares several optimization strategies on square matrices of different sizes.

2 Experimental Environment

- Operating system: Linux / WSL2
- Kernel: Linux ztw 6.6.87.2-microsoft-standard-WSL2 x86_64
- Compiler: cc (Ubuntu 13.3.0-6ubuntu2~24.04.1) 13.3.0
- Compiler flags: `-std=c11 -O3 -Wall -Wextra -pedantic`
- Parallel support: OpenMP
- BLAS library: OpenBLAS

The benchmark in this report was built and executed with:

```
make clean
make USE_OPENBLAS=1 USE_OPENMP=1
./benchmark > 1.out
```

3 Implemented Methods

The project includes five multiplication paths:

- `plain`: baseline triple-loop implementation
- `v1`: loop reordering from `ijk` to `ikj`
- `v2`: blocked multiplication with OpenMP on top of `ikj`
- `v3`: Strassen recursive multiplication
- `openblas`: `cblas_sgemv` as the external reference

All matrix elements use `float`.

4 Baseline Implementation: plain

The baseline version uses the standard triple nested loop:

```
for (i = 0; i < a->rows; ++i)
    for (j = 0; j < b->cols; ++j)
        for (k = 0; k < a->cols; ++k)
            c->data[i * c->cols + j] +=
                a->data[i * a->cols + k] * b->data[k * b->cols + j];
```

Its main drawback is poor cache locality when accessing $B[k][j]$ by column.

5 Improved Implementations

5.1 v1: Loop Reordering

The idea of v1 is to change the loop order from ijk to ikj . This allows the innermost loop to access one full row of B and one full row of C consecutively.

```
for (i = 0; i < a->rows; ++i) {
    const float *a_row = &a->data[i * a->cols];
    float *c_row = &c->data[i * c->cols];

    for (k = 0; k < a->cols; ++k) {
        const float a_ik = a_row[k];
        const float *b_row = &b->data[k * b->cols];

        for (j = 0; j < b->cols; ++j)
            c_row[j] += a_ik * b_row[j];
    }
}
```

Main effects of v1:

- more contiguous memory access for B and C
- fewer cache misses than the baseline
- more friendly to compiler auto-vectorization

This version is not a hand-written SIMD intrinsic implementation. However, it is SIMD-friendly. GCC vectorization reports show that the innermost j loop in v1 was automatically vectorized under `-O3 -fopenmp`:

```
matmul.c:91:27: optimized: loop vectorized using 16 byte vectors
matmul.c:91:27: optimized: loop versioned for vectorization because of
    possible aliasing
```

5.2 v2: Blocking + OpenMP

v2 adds cache blocking on top of the ikj traversal. The matrix is split into smaller tiles with block size 64.

```
for (i0 = 0; i0 < a->rows; i0 += MATMUL_BLOCK_SIZE) {
    size_t i1 = i0 + MATMUL_BLOCK_SIZE < a->rows ?
                i0 + MATMUL_BLOCK_SIZE : a->rows;

    for (k0 = 0; k0 < a->cols; k0 += MATMUL_BLOCK_SIZE) {
        size_t k1 = k0 + MATMUL_BLOCK_SIZE < a->cols ?
```

```

        k0 + MATMUL_BLOCK_SIZE : a->cols;

    for (j0 = 0; j0 < b->cols; j0 += MATMUL_BLOCK_SIZE) {
        size_t j1 = j0 + MATMUL_BLOCK_SIZE < b->cols ?
            j0 + MATMUL_BLOCK_SIZE : b->cols;

        for (i = i0; i < i1; ++i)
            for (k = k0; k < k1; ++k)
                for (j = j0; j < j1; ++j)
                    c_row[j] += a_ik * b_row[j];
    }
}

```

OpenMP is applied to the outer block loop:

```
#pragma omp parallel for schedule(static) private(j0, k0)
```

Main effects of v2:

- improved cache locality through blocking
- parallel execution through OpenMP
- much better performance on medium and large matrices

GCC vectorization reports also show auto-vectorization inside the block kernel:

```

matmul.c:134:40: optimized: loop vectorized using 16 byte vectors
matmul.c:134:40: optimized: loop versioned for vectorization because of
    possible aliasing

```

So the speedup of v2 comes from blocking, OpenMP, and compiler auto-vectorization.

5.3 v3: Strassen Recursive Algorithm

v3 uses the Strassen algorithm, which is an algorithm-level optimization rather than a simple loop optimization. A matrix is split into four submatrices:

$$A = \begin{bmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{bmatrix}, \quad B = \begin{bmatrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{bmatrix}$$

Strassen computes seven intermediate products:

$$\begin{aligned}
 M_1 &= (A_{11} + A_{22})(B_{11} + B_{22}) \\
 M_2 &= (A_{21} + A_{22})B_{11} \\
 M_3 &= A_{11}(B_{12} - B_{22}) \\
 M_4 &= A_{22}(B_{21} - B_{11}) \\
 M_5 &= (A_{11} + A_{12})B_{22} \\
 M_6 &= (A_{21} - A_{11})(B_{11} + B_{12}) \\
 M_7 &= (A_{12} - A_{22})(B_{21} + B_{22})
 \end{aligned}$$

Then the result blocks are reconstructed as:

$$\begin{aligned}
 C_{11} &= M_1 + M_4 - M_5 + M_7 \\
 C_{12} &= M_3 + M_5 \\
 C_{21} &= M_2 + M_4 \\
 C_{22} &= M_1 - M_2 + M_3 + M_6
 \end{aligned}$$

The implementation follows this structure directly. A representative code fragment is:

```

if ((rc = matrix_add(a11, a22, s1, 1)) != 0) goto cleanup;
if ((rc = matrix_add(b11, b22, s2, 1)) != 0) goto cleanup;
if ((rc = matmul_strassen_recursive(s1, s2, m1)) != 0) goto cleanup;
...
if ((rc = matrix_add(m1, m4, c11, 1)) != 0) goto cleanup;
if ((rc = matrix_add(c11, m5, c11, -1)) != 0) goto cleanup;
if ((rc = matrix_add(c11, m7, c11, 1)) != 0) goto cleanup;

```

Its theoretical complexity is lower than classical $O(n^3)$, but the actual implementation introduces heavy recursion, temporary matrix allocation, and data movement. Therefore, it does not necessarily outperform v2 in practice.

6 Correctness Check

According to 1.out, all small validation cases passed:

- plain vs v1: n=4,7,16,31 all PASS
- plain vs v2: n=4,7,16,31 all PASS
- plain vs v3: n=4,7,16,31 all PASS
- v3 vs OpenBLAS: n=4,7,16,31 all PASS

This shows that all three improved versions produced correct results in the tested cases, and v3 matched OpenBLAS as well.

7 Benchmark Results

7.1 Results for $n = 16$

Implementation	Time (s)	GFLOPS	Correctness
plain	0.000003	2.990	PASS
v1	0.000001	6.776	PASS
v2	0.000042	0.194	PASS
v3	0.000042	0.194	PASS
openblas	0.000000	16.483	PASS

7.2 Results for $n = 128$

Implementation	Time (s)	GFLOPS	Correctness
plain	0.001897	2.211	PASS
v1	0.000326	12.856	PASS
v2	0.000645	6.502	PASS
v3	0.000246	17.032	PASS
openblas	0.000038	110.009	PASS

7.3 Results for $n = 1024$

Implementation	Time (s)	GFLOPS	Correctness
plain	2.227743	0.964	PASS
v1	0.087690	24.490	PASS
v2	0.019736	108.811	PASS
v3	0.291508	7.367	PASS
openblas	0.002139	1004.035	PASS

7.4 Results for $n = 8192$

Implementation	Time (s)	GFLOPS	Correctness
plain	7013.361078	0.157	PASS
v1	113.915864	9.652	PASS
v2	8.246949	133.323	PASS
v3	66.821979	16.454	PASS
openblas	1.632326	673.586	PASS

7.5 Comparison of the Three Improved Versions

Size	v1 Time (s)	v1 GFLOPS	v2 Time (s)	v2 GFLOPS	v3 Time (s)	v3 GFLOPS
16	0.000001	6.776	0.000042	0.194	0.000042	0.194
128	0.000326	12.856	0.000645	6.502	0.000246	17.032
1024	0.087690	24.490	0.019736	108.811	0.291508	7.367
8192	113.915864	9.652	8.246949	133.323	66.821979	16.454

7.6 Result for $n = 65536$

Implementation	Result
All implementations	allocation failed

The output reports an estimated working set of 65536.00 MiB, which could not be allocated in the current environment.

8 Result Analysis

1. The baseline `plain` version is always the slowest one. Its performance is especially poor for 1024 and 8192, which confirms that the original triple loop has poor cache locality on large matrices.
2. `v1` is clearly faster than `plain`, showing that loop reordering alone already improves memory access behavior.
3. `v2` is the best self-implemented version in this project. It reaches 108.811 GFLOPS at 1024 and 133.323 GFLOPS at 8192, much faster than `plain`, `v1`, and `v3`.
4. `v3` uses the Strassen recursive algorithm. Although its theoretical complexity is better, the current implementation suffers from recursive overhead, temporary matrix allocation, and extra matrix addition/subtraction. As a result, it is slower than `v2` in practice.

5. The comparison table of `v1`, `v2`, and `v3` shows that small-size results fluctuate more, but for representative sizes such as 1024 and 8192, `v2` is clearly the best among the three self-implemented optimized versions.
6. OpenBLAS is faster than every self-implemented version at all meaningful sizes, which is expected because it uses mature packing, vectorization, and threading strategies.

9 Conclusion

This project implemented four matrix multiplication methods: `plain`, `v1`, `v2`, and `v3`, and compared them with OpenBLAS.

The benchmark results show:

- `v1` confirms that loop reordering is an effective optimization.
- `v2` demonstrates that blocking is the most successful optimization in this project for medium and large matrices.
- `v3`, although based on the Strassen algorithm, does not outperform `v2` in the current implementation.
- OpenBLAS remains the fastest overall implementation.

Therefore, if the criterion is the best self-implemented method, the best result in this project is `v2`. If the criterion is overall peak performance, OpenBLAS is the best performer.